Arboles generales y binarios.

Teoría:

Un árbol general es una estructura de datos no lineal que organiza elementos en forma jerárquica mediante nodos conectados entre si cada nodo puede tener uno, cero o muchos hijos Teniendo un nodo **Raíz único**

Un árbol binario comparte la jerarquía sin ciclos pese a esto existen diferencias prácticas que debemos tener presentes desde el diseño:

Número de enlaces: Los nodos binarios fijan dos punteros (izquierdo y derecho) mientras que el nodo general maneja arreglos, listas o esquema hijo- izquierdo / hermano-derecho) para sostener cualquier cantidad de hijos

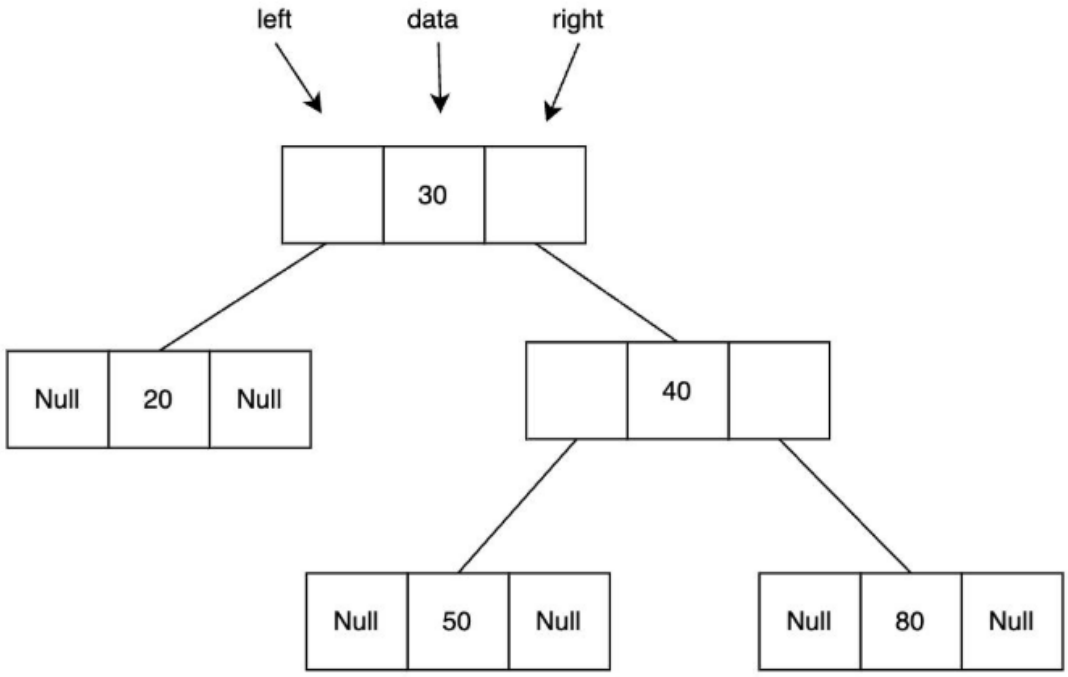
Recorridos: Los recorridos clásicos (inordenn. preorden, postorden) requieren ajustes porque el orden de hijos ya no esté atado a la izquierda y derecha si no a una secuencia de lista

Operaciones de inserción: Agregar un hijo nuevo a un árbol general implica decidir su posicion relativa respecto a los demás hermanos, algo que en binarios no existe por que la ubicación depende si o si, si es izquierdo o derecho

Aplicaciones: un árbol binario suele ser ideal para búsqueda y ordenamiento; uno general presenta mejores estructuras cuyo número de ramas varia: taxonomías, escenas, directorios, modelos de decisiones complejos y mas

Estructura de los datos en Arboles binarios

- **Raíz (*Root*):** El nodo inicial o principal en la parte superior del árbol.
- **Nodo padre (*Parent*):** Cualquier nodo que tiene uno o más descendientes.
- **Nodo hijo (*Child*):** Nodo que desciende de un nodo padre.
- **Nodos hermanos (*Siblings*):** Nodos que comparten el mismo nodo padre.
- **Nodos hojas (*Leaves*):** Nodos que no tienen hijos (apuntan a `null`).
- **Altura (*Height*):** El número de niveles o la distancia máxima desde la raíz hasta el nodo hoja más lejano




```
#include <iostream>
using namespace std;

struct Nodo {
    int dato;
    Nodo* izquierdo;
    Nodo* derecho;
};
    // Crear la raíz
    Nodo* raiz = new Nodo{10, nullptr, nullptr};

    // Crear los hijos
    raiz->izquierdo = new Nodo{20, nullptr, nullptr};
    raiz->derecho = new Nodo{30, nullptr, nullptr};
```

```
    return 0;
}
```

Recorridos de un árbol:

Preorden (raiz – izquierda – derecha)

Este recorrido comienza visitando la raíz del árbol. Después, recorre completamente el subárbol izquierdo y, finalmente, el subárbol derecho.

Su característica principal es que cada nodo se visita antes que sus hijos.

Inorden (izquierda-raiz-derecha)

Este recorrido comienza visitando el subárbol izquierdo. Una vez terminado, visita la raíz y continúa con el subárbol derecho.

Su característica principal es que la raíz se visita después del subárbol izquierdo y antes del derecho.

Postorden
```
// Hijos del nodo 50
insertarIzquierdo(arbol->izquierdo, 80);
insertarDerecho(arbol->izquierdo, 100);

// Hijos del nodo 10
insertarIzquierdo(arbol->derecho, 5);
insertarDerecho(arbol->derecho, 70);
```

(izquierda – derecha -raiz)

Este recorrido comienza visitando el subárbol izquierdo, continúa con el subárbol derecho y termina visitando la raíz.

Su característica principal es que cada nodo se visita después de haber recorrido a sus hijos.